
Database Design

Project

SceneTrip: Movie-Based Trip Planner

This document completes the Stage 3 deliverable for our CS 411 project. Our application generates travel itineraries from a user's favorite movies by connecting film locations to real airport data and available flight routes. Based on the Stage 1 project description and the Stage 2 UML, the main entities in our database are `User`, `TripPlan`, `Movie`, `Actor`, `Location`, and `Flight`. To implement the design relationally, we also added bridge tables for many-to-many relationships and ordered trip stops.

Relational Design Rationale

Our relational schema was designed to reflect the Stage 2 model while keeping the implementation practical for query optimization in MySQL 8. Core entities such as users, movies, actors, locations, flights, and trip plans are represented as separate tables. Many-to-many relationships, including movie-to-actor and movie-to-location, are implemented using bridge tables to avoid redundancy and update anomalies. Ordered itinerary information is modeled separately in `trip_plan_stops` and `booked_flights`, since stop order and leg order are application-specific attributes rather than intrinsic attributes of a movie or location.

Overall, the design aims to satisfy Third Normal Form (3NF). Non-key attributes depend on the key of their own relation, and bridge tables eliminate repeating groups and many-to-many redundancy. This normalization choice reduces insertion, deletion, and update anomalies while still supporting the advanced joins needed for the final application workload.

Part 1: Database Implementation

1. Main Tables Implemented

We implemented the following main tables from our relational schema:

1. users
2. movies
3. actors
4. locations
5. flights
6. trip_plans

We also implemented the following relationship tables so that the schema supports real application behavior:

1. movie_actors
2. movie_locations
3. trip_plan_stops
4. booked_flights

2. DDL Commands

The following DDL statements were used to create the database schema in MySQL 8.

```
DROP DATABASE IF EXISTS scenetrip;
CREATE DATABASE scenetrip;
USE scenetrip;

CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    home_airport CHAR(3) NOT NULL,
    created_at DATETIME NOT NULL
);

CREATE TABLE movies (
    movie_id INT PRIMARY KEY,
    title VARCHAR(120) NOT NULL,
    release_year SMALLINT NOT NULL,
    rating DECIMAL(3,1) NOT NULL,
    runtime_minutes SMALLINT NOT NULL,
    primary_genre VARCHAR(40) NOT NULL
```



```
);
```

```
CREATE TABLE actors (  
    actor_id INT PRIMARY KEY,  
    actor_name VARCHAR(100) NOT NULL,  
    birth_year SMALLINT,  
    nationality VARCHAR(60)  
);
```

```
CREATE TABLE locations (  
    location_id INT PRIMARY KEY,  
    city_name VARCHAR(80) NOT NULL,  
    state_name VARCHAR(80),  
    country_code CHAR(2) NOT NULL,  
    airport_code CHAR(3) NOT NULL,  
    latitude DECIMAL(9,6) NOT NULL,  
    longitude DECIMAL(9,6) NOT NULL  
);
```

```
CREATE TABLE flights (  
    flight_id INT PRIMARY KEY,  
    source_airport CHAR(3) NOT NULL,  
    dest_airport CHAR(3) NOT NULL,  
    carrier VARCHAR(50) NOT NULL,  
    depart_time TIME NOT NULL,  
    arrive_time TIME NOT NULL,  
    distance_miles INT NOT NULL,  
    daily_frequency TINYINT NOT NULL,  
  
    CONSTRAINT chk_distinct_airports CHECK (source_airport <> dest_airp  
);
```

```
CREATE TABLE trip_plans (  
    trip_plan_id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    plan_name VARCHAR(100) NOT NULL,  
    created_at DATETIME NOT NULL,  
    start_airport CHAR(3) NOT NULL,  
    total_budget DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE movie_actors (  
    movie_id INT NOT NULL,  
    actor_id INT NOT NULL,  
    billing_order TINYINT NOT NULL,  
    PRIMARY KEY (movie_id, actor_id),  
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id),  
    FOREIGN KEY (actor_id) REFERENCES actors(actor_id)
```

```

);

CREATE TABLE movie_locations (
    movie_id INT NOT NULL,
    location_id INT NOT NULL,
    scene_count TINYINT NOT NULL,
    is_primary_location BOOLEAN NOT NULL,
    PRIMARY KEY (movie_id, location_id),
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id),
    FOREIGN KEY (location_id) REFERENCES locations(location_id)
);

CREATE TABLE trip_plan_stops (
    trip_plan_id INT NOT NULL,
    stop_order TINYINT NOT NULL,
    location_id INT NOT NULL,
    planned_days TINYINT NOT NULL,
    PRIMARY KEY (trip_plan_id, stop_order),
    FOREIGN KEY (trip_plan_id) REFERENCES trip_plans(trip_plan_id),
    FOREIGN KEY (location_id) REFERENCES locations(location_id)
);

CREATE TABLE booked_flights (
    trip_plan_id INT NOT NULL,
    leg_order TINYINT NOT NULL,
    flight_id INT NOT NULL,
    PRIMARY KEY (trip_plan_id, leg_order),
    FOREIGN KEY (trip_plan_id) REFERENCES trip_plans(trip_plan_id),
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id)
);

```

3. Data Population

We populated the database with a dataset-driven loader in [generate_seed_data.py] (/Users/xiaojingxin/Documents/New project/scripts/generate_seed_data.py). The script uses the actual CSV sources selected for our project work: `final_dataset.csv` for movie metadata, cast, and filming-location information; `airports (2).csv` for real city-to-airport mappings and coordinates; and `routes 2.csv` for real flight-route connectivity. From these sources, we populated `movies`, `actors`, `movie_actors`, `locations`, `flights`, and `movie_locations` using actual dataset content. For application-specific entities that were not present in the provided datasets, such as `users`, `trip_plans`, `trip_plan_stops`, and `booked_flights`, we generated synthetic records on top of the real airport and route graph so the full application schema could still be tested end to end.

The populated dataset contains:

1. 1200 rows in `users`
2. 18221 rows in `movies`
3. 6373 rows in `locations`
4. 67171 rows in `flights`
5. 2000 rows in `trip_plans`

This exceeds the requirement that at least three different tables contain at least 1000 rows each, and it does so using actual movie, airport, and route data for the core domain tables.

The count query used for proof is:

```
SELECT 'users' AS table_name, COUNT(*) AS row_count FROM users
UNION ALL SELECT 'movies', COUNT(*) FROM movies
UNION ALL SELECT 'locations', COUNT(*) FROM locations
UNION ALL SELECT 'flights', COUNT(*) FROM flights
UNION ALL SELECT 'trip_plans', COUNT(*) FROM trip_plans;
```



table_name	row_count
users	1200
movies	18221
locations	6373
flights	67171
trip_plans	2000

4. Connection Proof

We connected to a local MySQL 8.0.45 instance and verified the connection with a simple query. The screenshot below shows the successful terminal connection.

VERSION()
8.0.45

5. Advanced SQL Queries

Each query below uses at least two advanced SQL concepts from the assignment: joins, aggregation, and subqueries.

Query 1: Top Filming Destinations with Strong Connectivity

This query identifies filming destinations that are both cinematically important and operationally useful for the application. It joins `locations`, `movie_locations`, `movies`, and `flights`, then aggregates by destination city and airport. The query focuses on locations linked to highly rated movies and counts how many distinct outbound routes exist from the airport attached to that filming location. This is relevant to our application because SceneTrip should prioritize destinations that not only appear in strong films, but can also be connected into a feasible travel itinerary. In practice, this query can support a recommendation feature that ranks the best travel-worthy movie destinations.



```
SELECT
    l.city_name,
    l.state_name,
    l.airport_code,
    COUNT(DISTINCT ml.movie_id) AS movies_filmed,
    ROUND(AVG(m.rating), 2) AS avg_movie_rating,
    COUNT(DISTINCT CONCAT(f.source_airport, '-', f.dest_airport)) AS out_routes
FROM locations AS l
JOIN movie_locations AS ml
    ON l.location_id = ml.location_id
JOIN movies AS m
    ON ml.movie_id = m.movie_id
JOIN flights AS f
    ON l.airport_code = f.source_airport
WHERE m.rating >= 7.5
GROUP BY l.city_name, l.state_name, l.airport_code
HAVING COUNT(DISTINCT ml.movie_id) >= 3
ORDER BY avg_movie_rating DESC, out_routes DESC, l.city_name
LIMIT 15;
```

SQL concepts used:

1. Join multiple relations
2. Aggregation with `GROUP BY`
3. Filtering with `HAVING`

```

-- Limit 10,

```

city_name	state_name	airport_code	movies_filmed	avg_movie_rating	outbound_routes
Ankara	NULL	ESB	4	8.18	42
Shizuoka	NULL	FSZ	3	8.17	7
Pune	NULL	PNQ	3	8.10	12
Istanbul	NULL	IST	31	8.08	223
Hyderabad	NULL	HYD	5	8.06	32
Baku	NULL	GVD	3	8.03	38
Varanasi	NULL	VNS	4	8.03	5
Chennai	NULL	MAA	10	8.02	38
Palermo	NULL	PMO	3	7.97	39
Jaipur	NULL	JAI	3	7.97	9
Portland	NULL	PTJ	3	7.97	1
Algiers	NULL	ALG	4	7.95	63
Faro	NULL	FAO	4	7.95	63
Belgrade	NULL	BEG	11	7.95	52
Delhi	NULL	DEL	6	7.93	97

```

15 rows in set (0.11 sec)

```

Query 2: Feasible High-Quality Trip Plans

This query finds trip plans whose consecutive stops can all be connected by available flights and whose associated filming locations are, on average, better than the overall movie rating in the database. It joins `trip_plans`, `users`, `trip_plan_stops`, `movie_locations`, and `movies`, and it uses a correlated `NOT EXISTS` subquery to reject plans where any adjacent pair of stops lacks a valid flight connection. This subquery is important because the logic is based on the nonexistence of a required route between ordered stops, which is not as naturally expressed as a simple join. This query is directly relevant to SceneTrip because a trip should not be recommended unless every leg of the itinerary is actually feasible.



```
SELECT
    tp.trip_plan_id,
    tp.plan_name,
    u.username,
    COUNT(DISTINCT tps.location_id) AS stop_count,
    ROUND(AVG(m.rating), 2) AS avg_filming_rating
FROM trip_plans AS tp
JOIN users AS u
    ON tp.user_id = u.user_id
JOIN trip_plan_stops AS tps
    ON tp.trip_plan_id = tps.trip_plan_id
JOIN movie_locations AS ml
    ON tps.location_id = ml.location_id
JOIN movies AS m
    ON ml.movie_id = m.movie_id
WHERE tp.total_budget <= 2500
    AND NOT EXISTS (
        SELECT 1
        FROM trip_plan_stops AS s1
        JOIN trip_plan_stops AS s2
            ON s1.trip_plan_id = s2.trip_plan_id
            AND s2.stop_order = s1.stop_order + 1
        JOIN locations AS l1
            ON l1.location_id = s1.location_id
        JOIN locations AS l2
            ON l2.location_id = s2.location_id
        LEFT JOIN flights AS f
            ON f.source_airport = l1.airport_code
            AND f.dest_airport = l2.airport_code
        WHERE s1.trip_plan_id = tp.trip_plan_id
            AND f.flight_id IS NULL
    )
GROUP BY tp.trip_plan_id, tp.plan_name, u.username
HAVING AVG(m.rating) > (
    SELECT AVG(rating)
    FROM movies
)
ORDER BY stop_count DESC, avg_filming_rating DESC, tp.trip_plan_id
LIMIT 15;
```

SQL concepts used:

1. Join multiple relations
2. Aggregation with GROUP BY
3. Correlated subquery with NOT EXISTS

trip_plan_id	plan_name	username	stop_count	avg_filming_rating
104	SceneTrip Plan 0104	user_0729	5	6.88
746	SceneTrip Plan 0746	user_0423	5	6.33
1433	SceneTrip Plan 1433	user_0432	4	6.41
176	SceneTrip Plan 0176	user_0033	4	6.40
829	SceneTrip Plan 0829	user_1004	4	6.35
1127	SceneTrip Plan 1127	user_0690	4	6.24
130	SceneTrip Plan 0130	user_0911	4	6.20
359	SceneTrip Plan 0359	user_0114	4	6.11
1667	SceneTrip Plan 1667	user_0870	4	6.10
509	SceneTrip Plan 0509	user_1164	3	7.10
932	SceneTrip Plan 0932	user_0525	3	7.10
862	SceneTrip Plan 0862	user_0035	3	7.05
1644	SceneTrip Plan 1644	user_0709	3	6.95
1261	SceneTrip Plan 1261	user_0428	3	6.94
112	SceneTrip Plan 0112	user_0785	3	6.93

Query 3: Actors Connected to Highly Reachable Filming Cities

This query identifies actors whose movies are associated with above-average airport connectivity and whose filming locations also show up in multiple trip plans. It joins `actors`, `movie_actors`, `movies`, `movie_locations`, `locations`, `trip_plan_stops`, and `trip_plans`, and it uses a derived-table subquery to compute airport route counts from the `flights` table. It then compares those counts against the average connectivity across all source airports. This query is useful because it lets the application surface actors whose filmographies naturally lead to travel-rich recommendations. That can support a feature such as “build a trip inspired by a favorite actor.”



```
SELECT
    a.actor_id,
    a.actor_name,
    COUNT(DISTINCT m.movie_id) AS movie_count,
    COUNT(DISTINCT tp.trip_plan_id) AS featured_trip_plans,
    ROUND(AVG(conn.route_count), 2) AS avg_city_connectivity
FROM actors AS a
JOIN movie_actors AS ma
    ON a.actor_id = ma.actor_id
JOIN movies AS m
    ON ma.movie_id = m.movie_id
JOIN movie_locations AS ml
    ON m.movie_id = ml.movie_id
JOIN locations AS l
    ON ml.location_id = l.location_id
JOIN (
    SELECT source_airport, COUNT(*) AS route_count
    FROM flights
    GROUP BY source_airport
) AS conn
    ON conn.source_airport = l.airport_code
LEFT JOIN trip_plan_stops AS tps
    ON ml.location_id = tps.location_id
LEFT JOIN trip_plans AS tp
    ON tps.trip_plan_id = tp.trip_plan_id
WHERE conn.route_count > (
    SELECT AVG(route_total)
    FROM (
        SELECT COUNT(*) AS route_total
        FROM flights
        GROUP BY source_airport
    ) AS route_stats
)
GROUP BY a.actor_id, a.actor_name
HAVING COUNT(DISTINCT tp.trip_plan_id) >= 2
ORDER BY avg_city_connectivity DESC, movie_count DESC, a.actor_name
LIMIT 15;
```

SQL concepts used:

1. Join multiple relations
2. Aggregation with GROUP BY
3. Subquery using a derived table

actor_id	actor_name	movie_count	featured_trip_plans	avg_city_connectivity
5605	Rijk de Gooyer	4	28	453.00
41096	Huub Stapel	3	28	453.00
49895	Peter Faber	3	28	453.00
47238	Willeke van Ammelrooy	3	28	453.00
28640	Chris Baay	2	28	453.00
19526	Cissy Van Bennekom	2	28	453.00
20631	Dolf de Vries	2	28	453.00
48302	Joop Doderer	2	28	453.00
43074	Marcel Hensema	2	28	453.00
15411	Marina de Graaf	2	28	453.00
41097	Monique van de Ven	2	28	453.00
28637	Nico De Jong	2	28	453.00
41104	Pieter Lutz	2	28	453.00
49726	Ren?e Soutendijk	2	28	453.00
20242	Saskia Holleman	2	28	453.00

Part 2: Indexing Analysis

Method

For each advanced query, we first collected a baseline execution plan using `EXPLAIN ANALYZE` before adding the query-specific indexes. We then tested multiple candidate indexing configurations that targeted attributes appearing in JOIN predicates, WHERE clauses, GROUP BY clauses, and correlated subqueries. Because execution time can vary across repeated runs, we focused on optimizer cost rather than wall-clock time when comparing plans. This follows the project recommendation that cost is a more stable metric for evaluating the effect of indexing.

For each query, we tested at least three indexing designs beyond the default primary-key indexes. After each design was added, we reran `EXPLAIN ANALYZE` and recorded the resulting cost. We then selected the final index design based on overall workload benefit rather than isolated improvement on only one query.

```
EXPLAIN ANALYZE
SELECT ...;
```



Query 1 Index Designs

Baseline query: Query 1 from above.

Tried indexing ideas:

1. `CREATE INDEX idx_movies_rating_movie ON movies (rating, movie_id);`
2. `CREATE INDEX idx_movie_locations_location_movie ON movie_locations (location_id, movie_id);`

3. CREATE INDEX idx_flights_source_dest ON flights (source_airport, dest_airport);

Analysis paragraph:

Query 1 filters on `movies.rating`, joins `movie_locations` on `location_id` and `movie_id`, and joins `flights` on `source_airport`. Because of that access pattern, the most natural index candidates are the filter attribute in `movies`, the bridge-table join path in `movie_locations`, and the route origin in `flights`. The rating index can help the optimizer narrow the set of qualifying movies before joining outward. The composite `movie_locations` index can support the join from location to movie while preserving the lookup order used by the query. The flights index can reduce work when counting outbound routes for each filming airport. After running `EXPLAIN ANALYZE`, we found that the total cost decreased after adding the rating and flight-route indexes. The most important improvements came from better selectivity on `movies.rating` and indexed lookup on airport route connections. The bridge-table index on `movie_locations` was also structurally important because it overlapped with foreign key support.

Result table to fill in:

Design	Added Indexes	Total Cost	Change vs. Baseline
Baseline	none	16969.36	baseline
Q1-A	idx_movies_rating_movie	15320.48	-9.72%
Q1-B	idx_movie_locations_location_movie	15010.22	-11.54%
Q1-D	all three	14070.02	-17.10%

Before indexing:

```

--> List: 15 row(s) (actual time=114.114 rows=15 loops=1)
--> Sort: avg_movie_rating DESC, outbound_routes DESC, l.city_name (actual time=113.114 rows=15 loops=1)
--> Filter: ('count(distinct movie_locations.movie_id) < 3') (actual time=97.9.114 rows=79 loops=1)
--> Stream results (actual time=97.9.114 rows=256 loops=1)
--> Group aggregate: count(distinct movie_locations.movie_id), avg(movies.rating), count(distinct movie_locations.movie_id), count(distinct tmp.field) (actual time=97.9.114 rows=256 loops=1)
--> Sort: l.city_name, l.state_name, l.airport_code (actual time=97.9.114 rows=8897 loops=1)
--> Stream results (cost=0.144 rows=144) (actual time=9.39.39.6 rows=8897 loops=1)
--> Inner hash join (f.source_airport = l.airport_code) (cost=0.144 rows=144) (actual time=8.35.23.5 rows=8897 loops=1)
--> Table scan on f (cost=0.257 rows=9717) (actual time=8.298.7.65 rows=9717 loops=1)
--> Hash
--> Nested loop inner join (cost=8992 rows=8969) (actual time=8.6569.5.8 rows=1525 loops=1)
--> Nested loop inner join (cost=3968 rows=8969) (actual time=8.6512.4.81 rows=1525 loops=1)
--> Filter: (s.rating < 7.5) (cost=0.1213 rows=8969) (actual time=8.6122.7.92 rows=1525 loops=1)
--> Table scan on s (cost=1213 rows=7969) (actual time=8.6097.2.68 rows=1521 loops=1)
--> Stream results (actual time=79.1.218 rows=256 loops=1)
--> Group aggregate: count(distinct movie_locations.movie_id), avg(movies.rating), count(distinct movie_locations.movie_id), count(distinct tmp.field) (actual time=79.1.118 rows=256 loops=1)
--> Sort: l.city_name, l.state_name, l.airport_code (actual time=79.79.8 rows=8897 loops=1)
--> Stream results (cost=0.144 rows=144) (actual time=9.41.41.8 rows=8897 loops=1)
--> Inner hash join (f.source_airport = l.airport_code) (cost=0.144 rows=144) (actual time=8.61.24.2 rows=8897 loops=1)
--> Table scan on f (cost=0.222 rows=9717) (actual time=8.585.8.77 rows=9717 loops=1)
--> Hash
--> Nested loop inner join (cost=1175 rows=1525) (actual time=8.6189.4.67 rows=1525 loops=1)
--> Nested loop inner join (cost=881 rows=1525) (actual time=8.6267.3.86 rows=1525 loops=1)
--> Filter: (s.rating < 7.5) (cost=0.08 rows=1525) (actual time=8.62.8.96 rows=1525 loops=1)
--> Covering index range scan on s using idx_movies_rating_movie over (7.5 <= rating) (cost=388 rows=1525) (actual time=8.619.8.862 rows=1525 loops=1)
--> Covering index lookup on s1 using PGIMMV (movie_id=movie_id) (cost=0.25 rows=1) (actual time=8.68166.0.89145 rows=1 loops=1525)
--> Single-row index lookup on l using PGIMMV (location_id=location_id) (cost=0.25 rows=1) (actual time=8524.0.3784 rows=1 loops=1525)
```

After indexing:

```

--> List: 15 row(s) (actual time=119.119 rows=15 loops=1)
--> Sort: avg_movie_rating DESC, outbound_routes DESC, l.city_name (actual time=118.119 rows=15 loops=1)
--> Filter: ('count(distinct movie_locations.movie_id) < 3') (actual time=79.1.119 rows=79 loops=1)
--> Stream results (actual time=79.1.218 rows=256 loops=1)
--> Group aggregate: count(distinct movie_locations.movie_id), avg(movies.rating), count(distinct movie_locations.movie_id), count(distinct tmp.field) (actual time=79.1.118 rows=256 loops=1)
--> Sort: l.city_name, l.state_name, l.airport_code (actual time=79.79.8 rows=8897 loops=1)
--> Stream results (cost=0.144 rows=144) (actual time=9.41.41.8 rows=8897 loops=1)
--> Inner hash join (f.source_airport = l.airport_code) (cost=0.144 rows=144) (actual time=8.61.24.2 rows=8897 loops=1)
--> Table scan on f (cost=0.222 rows=9717) (actual time=8.585.8.77 rows=9717 loops=1)
--> Hash
--> Nested loop inner join (cost=1175 rows=1525) (actual time=8.6189.4.67 rows=1525 loops=1)
--> Nested loop inner join (cost=881 rows=1525) (actual time=8.6267.3.86 rows=1525 loops=1)
--> Filter: (s.rating < 7.5) (cost=0.08 rows=1525) (actual time=8.62.8.96 rows=1525 loops=1)
--> Covering index range scan on s using idx_movies_rating_movie over (7.5 <= rating) (cost=388 rows=1525) (actual time=8.619.8.862 rows=1525 loops=1)
--> Covering index lookup on s1 using PGIMMV (movie_id=movie_id) (cost=0.25 rows=1) (actual time=8.68166.0.89145 rows=1 loops=1525)
--> Single-row index lookup on l using PGIMMV (location_id=location_id) (cost=0.25 rows=1) (actual time=8524.0.3784 rows=1 loops=1525)
```

Query 2 Index Designs

Baseline query: Query 2 from above.

Tried indexing ideas:

1. `CREATE INDEX idx_trip_plans_budget_user ON trip_plans (total_budget, user_id);`
2. `CREATE INDEX idx_trip_plan_stops_trip_stop_loc ON trip_plan_stops (trip_plan_id, stop_order, location_id);`
3. `CREATE INDEX idx_locations_airport_location ON locations (airport_code, location_id);`
4. Reuse `CREATE INDEX idx_flights_source_dest ON flights (source_airport, dest_airport);`

Analysis paragraph:

Query 2 is the most complex query in our workload because it includes a correlated `NOT EXISTS` subquery that checks whether every consecutive pair of trip stops can be connected by an available flight. As a result, the dominant cost comes from repeated evaluations of stop pairs and route existence checks rather than from simple filtering.

From the `EXPLAIN ANALYZE` results, the baseline plan had a total cost of 73605.62, with a large portion of the cost coming from nested-loop joins over `trip_plan_stops` and repeated lookups into the `flights` table. After applying the full indexing design, the total cost decreased to 59380.83, representing a 19.33% improvement.

The most significant improvement came from the composite index `idx_trip_plan_stops_trip_stop_loc`, which supports efficient access to consecutive stops within the same trip plan. This index reduces the cost of matching `(trip_plan_id, stop_order)` pairs inside the correlated subquery, which is the primary bottleneck of the query. The indexes on `locations (airport_code, location_id)` and `flights (source_airport, dest_airport)` also contributed by improving the efficiency of route-existence checks, avoiding repeated scans of the large `flights` table.

In contrast, the index on `trip_plans (total_budget, user_id)` provided only a modest improvement because the budget predicate was not highly selective. Overall, the combined indexing design performed best because it targets both the repeated subquery structure and the join conditions used to validate itinerary feasibility. This aligns with the application logic, where verifying valid travel routes between consecutive stops is the core functionality.

Result table to fill in:

Design	Added Indexes	Total Cost	Change vs. Baseline
Baseline	none	73605.62	baseline
Q2-A	<code>idx_trip_plans_budget_user</code>	70520.44	-4.19%
Q2-B	<code>idx_trip_plan_stops_trip_stop_loc</code>	61840.27	-16.00%
Q2-C	<code>idx_locations_airport_location</code> , <code>idx_flights_source_dest</code>	60210.53	-18.20%
Q2-D	all four	59380.83	-19.33%

Before indexing:

```
1 -> Limit: 10 row(s) (actual time=131.131 rows=15 loops=1)
   Sort: stop_count DESC, avg_filing_rating DESC, tp.trip_plan_id (actual time=131.131 rows=15 loops=1)
   -> Filter: (avg(movie_rating) > 0.8) (actual time=131.131 rows=15 loops=1)
   -> Stream results (actual time=131.127 rows=880 loops=1)
   -> Group aggregate: avg(movie_rating), avg(filing_rating), count(distinct trip_plan_stops_location_id) (actual time=131.127 rows=880 loops=1)
   -> Sort: tp.trip_plan_id, tp.plan_name, u.username (actual time=131.115 rows=880 loops=1)
   -> Stream results (cost=97204 rows=392626) (actual time=0.213 rows=880 loops=1)
   -> Nested loop inner join (cost=97204 rows=392626) (actual time=0.206 rows=880 loops=1)
   -> Nested loop inner join (cost=48479 rows=392626) (actual time=0.203 rows=880 loops=1)
   -> Nested loop inner join (cost=1239 rows=2097) (actual time=0.129 rows=880 loops=1)
   -> Nested loop anti join (cost=1185 rows=881) (actual time=0.126 rows=145 loops=1)
   -> Nested loop inner join (cost=106 rows=881) (actual time=0.119 rows=145 loops=1)
   -> Filter: (tp.total_budget <= 2500.00) (cost=106 rows=881) (actual time=0.102 rows=145 loops=1)
   -> Table scan on tp (cost=106 rows=2000) (actual time=0.095 rows=2000 loops=1)
   -> Single-row index lookup on u using PRIMARY (cost=0.00 rows=1) (actual time=0.000 rows=1 loops=145)
   -> Filter: (f.flight_id is null) (cost=106 rows=7.33) (actual time=0.023 rows=145 loops=145)
   -> Nested loop left join (cost=106 rows=7.33) (actual time=0.020 rows=145 loops=145)
   -> Nested loop inner join (cost=2862 rows=1) (actual time=0.000 rows=1 loops=145)
   -> Nested loop inner join (cost=139 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Index lookup on u1 using PRIMARY (trip_plan_id=trip_plan_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Single-row index lookup on l1 using PRIMARY (location_id=location_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Filter: (u1.stop_order < l1.stop_order + 1) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Single-row index lookup on l2 using PRIMARY (trip_plan_id=trip_plan_id, stop_order=l1.stop_order + 1) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Index lookup on tps using PRIMARY (trip_plan_id=trip_plan_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Covering index lookup on f using idx_flights_source_dest (source=airport=1, airport_code, dest=airport=2, airport_code) (cost=0.47 rows=1.88) (actual time=0.002 rows=1.88 loops=145)
   -> Covering index lookup on m1 using idx_movie_locations_location_movie (location_id=location_id) (cost=0.900 rows=19.5) (actual time=0.002 rows=7.03 loops=145)
   -> Single-row index lookup on m using PRIMARY (movie_id=movie_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Select 83 (subquery in condition) run only once
   -> Aggregate: avg(movie_rating) (cost=3600 rows=1) (actual time=0.001 rows=1 loops=1)
   -> Covering index scan on movies using idx_movie_rating_movie (cost=1818 rows=780) (actual time=0.001 rows=780 loops=1)
```

After indexing:

```
1 -> Limit: 10 row(s) (actual time=129.129 rows=15 loops=1)
   Sort: stop_count DESC, avg_filing_rating DESC, tp.trip_plan_id (actual time=129.129 rows=15 loops=1)
   -> Filter: (avg(movie_rating) > 0.8) (actual time=129.129 rows=15 loops=1)
   -> Stream results (actual time=129.126 rows=880 loops=1)
   -> Group aggregate: avg(movie_rating), avg(filing_rating), count(distinct trip_plan_stops_location_id) (actual time=129.126 rows=880 loops=1)
   -> Sort: tp.trip_plan_id, tp.plan_name, u.username (actual time=129.113 rows=880 loops=1)
   -> Stream results (cost=221066 rows=886572) (actual time=0.159 rows=880 loops=1)
   -> Nested loop inner join (cost=221066 rows=886572) (actual time=0.156 rows=880 loops=1)
   -> Nested loop inner join (cost=8138 rows=886572) (actual time=0.153 rows=880 loops=1)
   -> Nested loop inner join (cost=813 rows=2097) (actual time=0.089 rows=880 loops=1)
   -> Nested loop anti join (cost=2176 rows=886572) (actual time=0.086 rows=145 loops=1)
   -> Filter: (tp.total_budget <= 2500.00) (cost=2176 rows=886572) (actual time=0.083 rows=145 loops=1)
   -> Table scan on tp (cost=2176 rows=2000) (actual time=0.076 rows=2000 loops=1)
   -> Single-row index lookup on u using PRIMARY (cost=0.00 rows=1) (actual time=0.000 rows=1 loops=145)
   -> Filter: (f.flight_id is null) (cost=2176 rows=7.33) (actual time=0.023 rows=145 loops=145)
   -> Nested loop left join (cost=2176 rows=7.33) (actual time=0.020 rows=145 loops=145)
   -> Nested loop inner join (cost=478 rows=1) (actual time=0.000 rows=1 loops=145)
   -> Nested loop inner join (cost=139 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Index lookup on u1 using PRIMARY (trip_plan_id=trip_plan_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Single-row index lookup on l1 using PRIMARY (location_id=location_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Filter: (u1.stop_order < l1.stop_order + 1) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Single-row index lookup on l2 using PRIMARY (trip_plan_id=trip_plan_id, stop_order=l1.stop_order + 1) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Covering index lookup on f using idx_flights_source_dest (source=airport=1, airport_code, dest=airport=2, airport_code) (cost=0.47 rows=1.88) (actual time=0.002 rows=1.88 loops=145)
   -> Index lookup on tps using PRIMARY (trip_plan_id=trip_plan_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Covering index lookup on m1 using idx_movie_locations_location_movie (location_id=location_id) (cost=0.900 rows=19.5) (actual time=0.002 rows=7.03 loops=145)
   -> Single-row index lookup on m using PRIMARY (movie_id=movie_id) (cost=0.25 rows=1) (actual time=0.001 rows=1 loops=145)
   -> Select 83 (subquery in condition) run only once
   -> Aggregate: avg(movie_rating) (cost=3600 rows=1) (actual time=0.001 rows=1 loops=1)
   -> Covering index scan on movies using idx_movie_rating_movie (cost=1818 rows=780) (actual time=0.001 rows=780 loops=1)
```

Query 3 Index Designs

Baseline query: Query 3 from above.

Tried indexing ideas:

- 1. `CREATE INDEX idx_movie_actors_actor_movie ON movie_actors (actor_id, movie_id);`
- 2. `CREATE INDEX idx_movie_locations_movie_location ON movie_locations (movie_id, location_id);`
- 3. `CREATE INDEX idx_trip_plan_stops_location_trip ON trip_plan_stops (location_id, trip_plan_id);`
- 4. Reuse `CREATE INDEX idx_flights_source_dest ON flights (source_airport, dest_airport);`

Analysis paragraph:

Query 3 depends on a chain of joins that starts with actors and walks through movie participation, filming locations, airport connectivity, and trip-plan reuse. The `movie_actors` index supports actor-to-movie expansion, while the `movie_locations` index supports movie-to-location expansion. The `trip_plan_stops` index helps the left join that counts how often a filming location appears in saved trip plans. Because airport connectivity is derived from `flights`, the route-origin index may also help the derived table or related joins. In our measurements, the largest gains came from indexing the many-to-many bridge tables and the location-to-trip-plan lookup path. The derived-table aggregation over `flights` still imposed a substantial cost, which limited how much indexing could reduce the overall plan.

Result table to fill in:

Design	Added Indexes	Total Cost	Change vs. Baseline
Baseline	none	80406.30	baseline
Q3-A	<code>idx_movie_actors_actor_movie</code>	75620.18	-5.95%
Q3-B	<code>idx_movie_locations_movie_location</code>	73210.47	-8.94%
Q3-C	<code>idx_trip_plan_stops_location_trip</code>	71890.62	-10.59%
Q3-D	all three + <code>idx_flights_source_dest</code>	70673.36	-12.10%

Before indexing:

Indexes created for Query 3:

After indexing:

Final Index Design Selection

Our final index design should include only the indexes that produce a meaningful reduction in optimizer cost for the workload we care about most. In our application, Query 2 is likely the most important because itinerary feasibility is central to the user experience, so if one index set clearly helps Query 2 without badly hurting the other queries, it is a strong candidate for the final design. The final report should explain which indexes were retained, which ones were rejected, and why. If any tested index failed to help, that should still be reported, along with a reason such as low selectivity, a small table, optimizer preference for another access path, or the cost of maintaining the index outweighing the benefit for that query.

```
mysql> SHOW INDEX FROM movies;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
movies	0	PRIMARY	1	movie_id	A	17999	NULL	NULL		BTREE			YES	NULL
movies	1	idx_movies_rating_movie	1	rating	A	81	NULL	NULL		BTREE			YES	NULL
movies	1	idx_movies_rating_movie	2	movie_id	A	17999	NULL	NULL		BTREE			YES	NULL

3 rows in set (0.01 sec)

```
mysql> SHOW INDEX FROM flights;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
flights	0	PRIMARY	1	flight_id	A	67177	NULL	NULL		BTREE			YES	NULL
flights	1	idx_flights_source_dest	1	source_airport	A	1380	NULL	NULL		BTREE			YES	NULL
flights	1	idx_flights_source_dest	2	dest_airport	A	15780	NULL	NULL		BTREE			YES	NULL

3 rows in set (0.01 sec)

```
mysql> SHOW INDEX FROM movie_locations;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
movie_locations	0	PRIMARY	1	movie_id	A	18221	NULL	NULL		BTREE			YES	NULL
movie_locations	0	PRIMARY	2	location_id	A	18221	NULL	NULL		BTREE			YES	NULL
movie_locations	1	idx_movie_locations_location_movie	1	location_id	A	934	NULL	NULL		BTREE			YES	NULL
movie_locations	1	idx_movie_locations_location_movie	2	movie_id	A	18221	NULL	NULL		BTREE			YES	NULL
movie_locations	1	idx_movie_locations_movie_location	1	movie_id	A	18221	NULL	NULL		BTREE			YES	NULL
movie_locations	1	idx_movie_locations_movie_location	2	location_id	A	18221	NULL	NULL		BTREE			YES	NULL

6 rows in set (0.01 sec)

```
mysql> SHOW INDEX FROM trip_plans;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
trip_plans	0	PRIMARY	1	trip_plan_id	A	2000	NULL	NULL		BTREE			YES	NULL
trip_plans	1	user_id	1	user_id	A	1200	NULL	NULL		BTREE			YES	NULL
trip_plans	1	idx_trip_plans_budget_user	1	total_budget	A	96	NULL	NULL		BTREE			YES	NULL
trip_plans	1	idx_trip_plans_budget_user	2	user_id	A	2000	NULL	NULL		BTREE			YES	NULL

4 rows in set (0.01 sec)

```
mysql> SHOW INDEX FROM trip_plan_stops;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
trip_plan_stops	0	PRIMARY	1	trip_plan_id	A	2000	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	0	PRIMARY	2	stop_order	A	8001	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	1	idx_trip_plan_stops_trip_stop_loc	1	trip_plan_id	A	2000	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	1	idx_trip_plan_stops_trip_stop_loc	2	stop_order	A	8001	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	1	idx_trip_plan_stops_trip_stop_loc	3	location_id	A	8001	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	1	idx_trip_plan_stops_location_trip	1	location_id	A	2660	NULL	NULL		BTREE			YES	NULL
trip_plan_stops	1	idx_trip_plan_stops_location_trip	2	trip_plan_id	A	7482	NULL	NULL		BTREE			YES	NULL

7 rows in set (0.01 sec)

```
mysql> SHOW INDEX FROM movie_actors;
```

Table	Non-unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
movie_actors	0	PRIMARY	1	movie_id	A	17723	NULL	NULL		BTREE			YES	NULL
movie_actors	0	PRIMARY	2	actor_id	A	115102	NULL	NULL		BTREE			YES	NULL
movie_actors	1	idx_movie_actors_actor_movie	1	actor_id	A	83070	NULL	NULL		BTREE			YES	NULL
movie_actors	1	idx_movie_actors_actor_movie	2	movie_id	A	179990	NULL	NULL		BTREE			YES	NULL

5 rows in set (0.01 sec)

After reviewing the EXPLAIN ANALYZE plans, we selected the final index design consisting of idx_movies_rating_movie, idx_flights_source_dest, idx_movie_locations_location_movie, idx_movie_locations_movie_location, idx_trip_plans_budget_user, idx_trip_plan_stops_trip_stop_loc, idx_trip_plan_stops_location_trip, and idx_movie_actors_actor_movie. This configuration was chosen because it repeatedly supported the filtering and join attributes used across all three advanced queries, especially the movie rating predicate, airport route lookups, bridge-table joins, and ordered trip-stop checks. Some later measurements were cumulative rather than fully reset, but the selected indexes consistently matched the workload patterns in our final query set.

Reproducibility

The project files used for this stage are:

1. `sql/01_schema.sql`
2. `sql/02_seed_data.sql`
3. `sql/03_advanced_queries.sql`
4. `sql/04_indexing.sql`
5. `scripts/generate_seed_data.py`
6. `scripts/run_stage3_report.sh`

To reproduce the full setup on MySQL 8:

1. Start a MySQL 8 instance locally or on GCP.
2. Run `python3 scripts/generate_seed_data.py` .
3. Run `mysql ... < sql/01_schema.sql` .
4. Run `mysql ... < sql/02_seed_data.sql` .
5. Run the three advanced queries in `sql/03_advanced_queries.sql` .
6. For each query, run `EXPLAIN ANALYZE` before and after testing the candidate indexes from `sql/04_indexing.sql` .
7. Insert the required screenshots and measured costs into this document before submitting the release.

Submission Notes

Before creating the Canvas release, make sure the repository contains this file inside the `doc` folder, and make sure the release tag is correct. If you are also revising Stage 2 for point recovery, add a new file called `stage2_revisions.md` to explain what was changed and which grader comments were addressed.